

OpenMP Tutorial

Taura-Lab Spring Training 2026

Yuki Maeda

Taura Lab, M1

2026.06.26

Goals of This Tutorial

- Understand the concepts of OpenMP
- Learn the basics of using OpenMP
- Learn more generally what to keep in mind in parallel programming

目次

▸ **What is OpenMP?**

Loop Parallelization

For Correct Parallelization

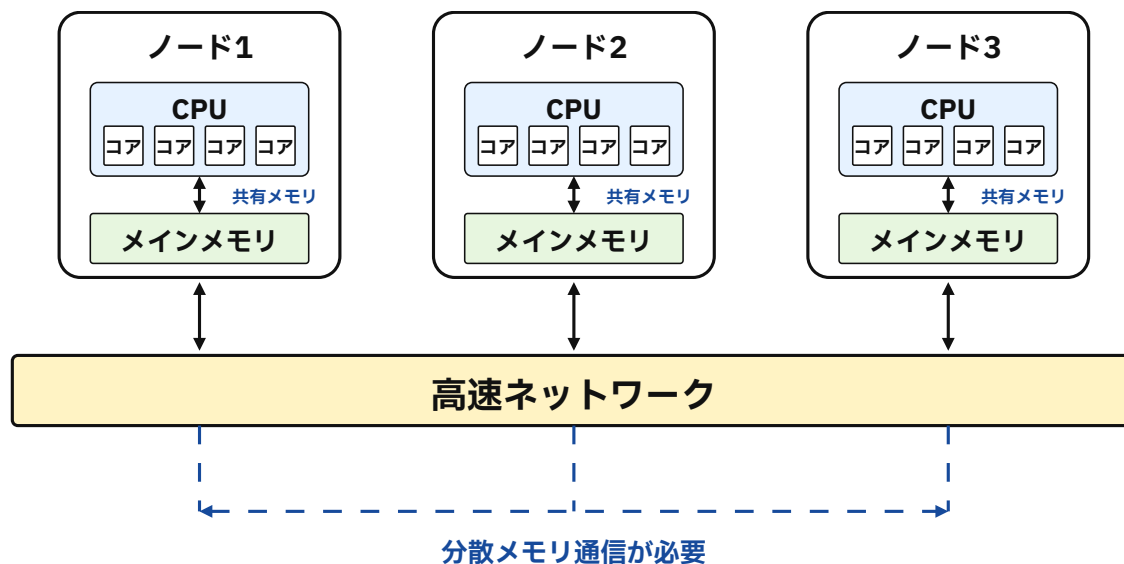
For Fast Parallelization

Other Constructs

Appendix

Basic Architecture of HPC Systems

- In HPC, compute clusters formed by connecting many machines over a network are commonly used
- Each machine in a cluster is called a **node**
- Each node has a CPU, memory, and sometimes a GPU
 - A CPU contains multiple **cores** that can execute multiple operations in parallel
- Within one node, multiple cores share memory (**shared memory**)
- Across nodes, memory is not shared (**distributed memory**)



What is OpenMP?

- **OpenMP** is an API specification and standard for parallelizing programs on shared-memory parallel computers
 - MP stands for Multi-Processing
 - Available from C, C++, and Fortran
- Advantage: parallel processing can be written easily using compiler directives such as `#pragma omp parallel`
 - **Parallelization can be added easily with minimal changes to existing sequential code**
- Example: parallelizing a for loop

```
for (int i=0; i<N; i++) {  
    a[i] = i;  
}
```



```
#pragma omp parallel for  
for (int i=0; i<N; i++) {  
    a[i] = i;  
}
```

[Note] What Does “API Specification / Standard” Mean?

- OpenMP is not the name of a single library, but a **specification** for parallelization
- The specification includes the following:
 - Compiler directives such as `#pragma omp ...`
 - Runtime functions such as `omp_get_thread_num()`
 - Environment variables such as `OMP_NUM_THREADS`
- In practice, compilers and runtimes implement this specification
 - GCC: `libgomp`
 - LLVM/Clang: `libomp`
 - Intel oneAPI: Intel OpenMP runtime
- OpenMP specification: <https://www.openmp.org/specifications/>

[Note] What is pragma?

- A pragma is a directive to the compiler (**directive**)
- In C/C++, it is written as a line starting with **#pragma**
- In OpenMP, parallelization instructions are given to the compiler in the form **#pragma omp ...**
- OpenMP uses pragmas so that parallelization directives can be added to existing C/C++/Fortran programs without significantly changing the original code structure

Intra-node and Inter-node Parallelization

- OpenMP handles intra-node parallelization (**shared-memory parallelization**)
- APIs such as **MPI** handle inter-node parallelization (**distributed-memory parallelization**)
- In real HPC, **hybrid parallelism** using MPI across nodes and OpenMP within nodes is important
 - OpenMP alone is mainly limited to a single node; MPI is needed to scale out for large computations
 - On the other hand, distributed memory requires attention to communication, data placement, and load balancing, and achieving good performance is not easy

#pragma omp parallel

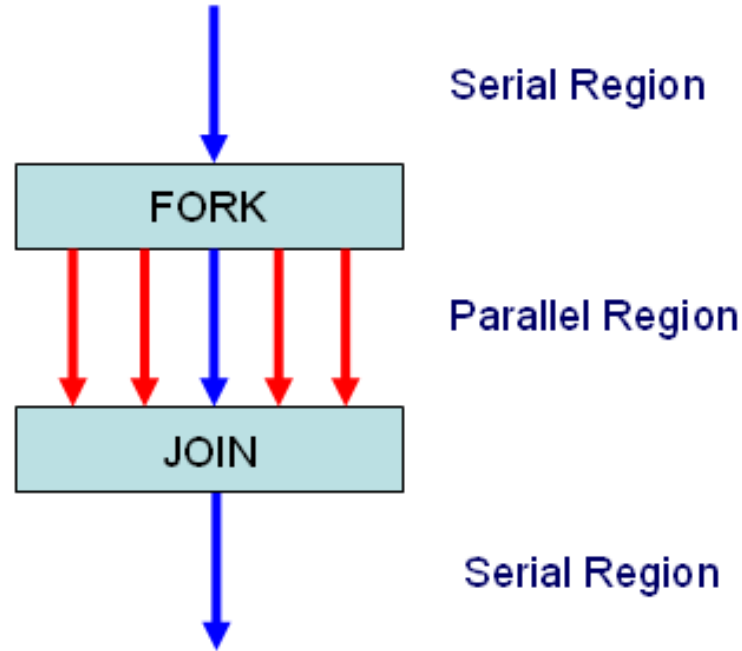
- A basic directive for creating a **parallel region** executed by multiple threads
- Written as below, each thread executes the statements in the parallel region
- Note that **the same statements are executed once per thread**
- Threads synchronize at the end of the block (**fork-join model**)

```
// sequential region

#pragma omp parallel
{
    // parallel region
}

// sequential region
```

Fork-Join Model



- Upon entering the parallel region, execution forks into multiple threads (**fork**)
- Each thread executes its work in parallel
- Threads synchronize at the end of the parallel region (**join**)

Source: <https://parallelcomputingsite.wordpress.com/2017/04/17/fork-join-in-open-mp/>

Sample Program: hello.c

```
printf("Before parallel region\n");
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int nthreads = omp_get_num_threads();
    printf("Hello, World! I'm thread %d of %d\n", tid, nthreads);
}
printf("After parallel region\n");
```

- Written this way, all threads execute the statements inside { }
- `omp_get_num_threads()` returns the total number of threads
- `omp_get_thread_num()` returns this thread's ID

Let's Try Compiling

- To use OpenMP, pass `-fopenmp` at compile time

```
$ gcc -fopenmp hello.c -o hello  
$ clang -fopenmp hello.c -o hello
```

- However, with clang, an OpenMP runtime may be required separately
- Note: on macOS this gets complicated
 - On macOS, gcc is only named gcc; it is actually Apple Clang
 - In that case, you cannot compile as shown above
 - The easiest approach is to install gcc via Homebrew

```
$ brew install gcc  
$ gcc-<your_gcc_version> -fopenmp hello.c -o hello
```

- For the sample programs, change the Makefile to `CC := gcc-<your_gcc_version>`

[Note] What is Apple Clang?

- On macOS, run the following:

```
$ gcc --version
Apple clang version 17.0.0 (clang-1700.6.3.2)
Target: arm64-apple-darwin25.1.0
Thread model: posix
InstalledDir: /Applications/Xcode.app/Contents/Developer/Toolchains/
XcodeDefault.xctoolchain/usr/bin
```

- Apple Clang does not directly support -fopenmp
- It is still possible to make Apple Clang work

```
$ brew install libomp
$ clang -Xpreprocessor -fopenmp -lomp \
-I$(brew --prefix libomp)/include \
-L$(brew --prefix libomp)/lib \
hello.c -o hello
```

Let's Try Running

- At runtime, you can specify the number of threads as follows:

```
$ OMP_NUM_THREADS=4 ./hello
```

- Alternatively, you can set it as an environment variable in advance

```
$ export OMP_NUM_THREADS=4  
$ ./hello
```

- You can also set it in the program with `omp_set_num_threads(num_threads)`

Execution Results

- Example output:

```
$ OMP_NUM_THREADS=4 ./hello
Before parallel region
Hello, World! I'm thread 1 of 4
Hello, World! I'm thread 2 of 4
Hello, World! I'm thread 0 of 4
Hello, World! I'm thread 3 of 4
After parallel region
```

- Indeed, each thread executes the statements inside the parallel region
- The order Before → Hello → After is always preserved
 - This is due to the fork-join model
 - The order within Hello may change on each run

In Practice?

- In practice, we want threads to share different work, not just execute the same code
- In OpenMP, `parallel` creates a team of threads, and constructs inside it split and distribute work
- Representative constructs:
 - **for construct**: `#pragma omp for`
splits loop iterations
 - **sections construct**: `#pragma omp sections`
runs multiple independent blocks in parallel
 - **task construct**: `#pragma omp task`
dynamically creates tasks for flexible distribution
- Note that `for` and `sections` also have combined short forms with `parallel`
 - `#pragma omp parallel for`
 - `#pragma omp parallel sections`
- `task` is usually created from `single` inside a `parallel` region

目次

What is OpenMP?

▸ **Loop Parallelization**

For Correct Parallelization

For Fast Parallelization

Other Constructs

Appendix

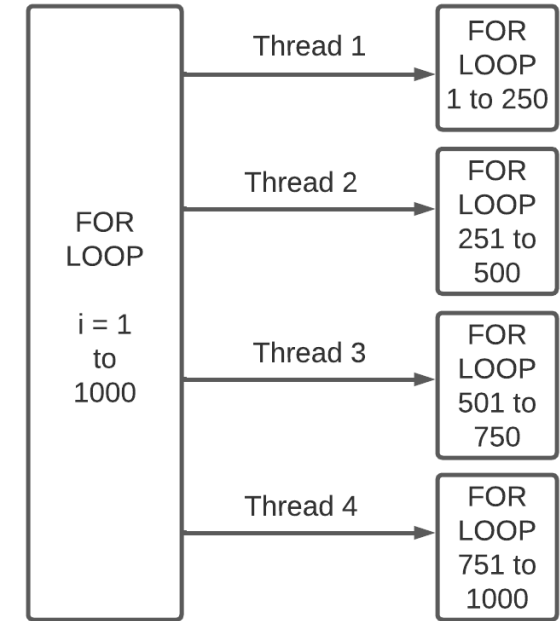
for Construct

- Written as below, each iteration of the for loop is distributed to threads

```
#pragma omp parallel  
{  
    #pragma omp for  
    for (int i = 0; i < N; i++) {  
        a[i] = i;  
    }  
}
```

- Can be abbreviated as follows

```
#pragma omp parallel for  
for (int i = 0; i < N; i++) {  
    a[i] = i;  
}
```



Source: <https://www.geeksforgeeks.org/c/c-parallel-for-loop-in-openmp/>

Let's Run parallel_for.c

- Let's run parallel_for.c
- Example output:

```
$ OMP_NUM_THREADS=4 ./parallel_for
Loop 2 is being processed by thread 1
Loop 3 is being processed by thread 1
Loop 4 is being processed by thread 2
Loop 5 is being processed by thread 2
Loop 0 is being processed by thread 0
Loop 1 is being processed by thread 0
Loop 6 is being processed by thread 3
Loop 7 is being processed by thread 3
```

- Loops are assigned to each thread in blocks of $\frac{\text{number of iterations}}{\text{number of threads}}$ starting from the lowest index
- This assignment method can be changed (discussed later)

目次

What is OpenMP?

Loop Parallelization

▸ **For Correct Parallelization**

▸ **Shared and Private Variables**

Examples That Cannot Be Parallelized

Data Races

For Fast Parallelization

Other Constructs

Appendix

Caution ①: Shared and Private Variables

- In OpenMP, variables are either shared across threads (**shared variables**) or allocated independently per thread (**private variables**)
 - This is called a variable's **attribute**
- Except for the loop variable immediately after `#pragma omp parallel for`, all variables are shared(!)
- That is, a program like the following does not work correctly
 - `j` is a shared variable
 - Between `j = i` and `a[j] = i`, `j` may be overwritten by another thread

```
int i, j;
#pragma omp parallel for
for (i = 0; i < N; i++) {
    j = i;
    a[j] = 1;
}
```

Caution ①: Shared and Private Variables

- Let's confirm that something actually goes wrong

```
#pragma omp parallel for
for (i = 0; i < 10000; i++) {
    j = i;
    a[j] = 1;
}

long sum = 0;
for (int idx = 0; idx < 10000; idx++) sum += a[idx];
```

- Output of attribution_err.c
 - The correct result is Sum = 10000

```
$ OMP_NUM_THREADS=4 ./attribution_err
Sum = 9986
$ OMP_NUM_THREADS=4 ./attribution_err
Sum = 9989
$ OMP_NUM_THREADS=1 ./attribution_err
Sum = 10000
```

Caution ①: Shared and Private Variables

- The correct approach: use a private clause to make variables private

```
int i, j;
#pragma omp parallel for private(j)
for (i = 0; i < N; i++) {
    j = i;
    a[j] = 1;
}
```

- Alternatively, default(private) makes private the default
- Or, in this case, this also works
 - Variables declared inside the parallel region should be private

```
int i;
#pragma omp parallel for
for (i = 0; i < N; i++) {
    int j = i;
    a[j] = 1;
}
```

[Note] Variable Attributes

- **private**: each thread gets its own copy; **the value at parallel region entry is undefined**
- **shared**: all threads refer to the same variable; conflicts require synchronization to avoid undefined behavior
- **firstprivate**: like private, but **initialized from the parent thread's value at parallel region entry**
- **lastprivate**: after the parallel region, the value from the last-updating thread is copied back to the parent thread
- See `attribution.c` for details

目次

What is OpenMP?

Loop Parallelization

▸ **For Correct Parallelization**

Shared and Private Variables

▸ **Examples That Cannot Be Parallelized**

Data Races

For Fast Parallelization

Other Constructs

Appendix

Caution ②: Examples That Cannot Be Parallelized

- Not every for loop can be parallelized
- Programs with **flow dependence**—where results differ unless i runs from 1 to N in order—are difficult to parallelize (unless the algorithm is changed)
- Example 1:

```
for (i = 1; i < N - 1; i++) {  
    a[i] = a[i] + 1;  
    b[i] = a[i - 1] + a[i + 1];  
}
```

- $a[i - 1]$ may not have been updated yet

Caution ②: Examples That Cannot Be Parallelized

- Example 2:

```
for (i = 0; i < N; i++) {  
    a[i] = a[b[i]];  
}
```

- Whether parallelization is possible depends on the contents of $b[i]$

目次

What is OpenMP?

Loop Parallelization

▸ **For Correct Parallelization**

Shared and Private Variables

Examples That Cannot Be Parallelized

▸ **Data Races**

For Fast Parallelization

Other Constructs

Appendix

Caution ③: Data Races

- Handling **data races** is important in parallel processing
- Example: estimating π with Monte Carlo
 - For each trial i , compute `trial_hits(i)` (throw arrows randomly SUBSAMPLES times) and add hits to the shared variable `hits`
- Below is an example that causes a data race

```
long long hits = 0;

#pragma omp parallel for
for (int i = 0; i < N; i++) {
    long long h = trial_hits(i);
    hits += h; // data race!
}
```

- Another thread may write to `hits` between the read and write
- OpenMP provides several ways to avoid data races

Avoiding Data Races ①: lock

- OpenMP provides lock mechanisms
- For the previous example:

```
omp_lock_t lock;  
omp_init_lock(&lock);  
#pragma omp parallel for  
for (int i = 0; i < N; i++) {  
    long long h = trial_hits(i);  
    omp_set_lock(&lock);  
    hits += h;  
    omp_unset_lock(&lock);  
}
```

Avoiding Data Races ②: critical Directive

- Statements inside `#pragma omp critical` are executed by at most one thread at a time
- Less flexible than locks, but simpler to implement
- Adding the optional **(name)** allows multiple critical sections
 - Critical sections with the same name exclude each other; different names can run concurrently
 - name is optional
- For the previous example:

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    long long h = trial_hits(i);
    #pragma omp critical
    {
        hits += h;
    }
}
```

Avoiding Data Races ③: atomic Directive

- For **a single simple expression** (such as `x+=value` or `x^=value`), `#pragma omp atomic` is faster
- It updates values efficiently using hardware atomic instructions
- For the previous example:

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    long long h = trial_hits(i);
    #pragma omp atomic
    hits += h;
}
```


Let's Compare

- Let's compare the execution times
 - Execution time can be measured with `omp_get_wtime()`
- Run `seq.c`, `lock.c`, `critical.c`, and `atomic.c`
 - `N = 20000000`, `SUBSAMPLES = 16`
- Example output:

```
$ ./seq
Elapsed time: 1.890952 seconds
$ ./lock
Elapsed time: 5.342480 seconds
$ ./critical
Elapsed time: 5.382293 seconds
$ ./atomic
Elapsed time: 1.273485 seconds
```

reduction Clause

- For combining results from multiple threads (e.g., a sum), a **reduction** clause is natural
- In the previous example, the addition to `hits` can be written as:

```
long long hits = 0;

#pragma omp parallel for reduction(+:hits)
for (int i = 0; i < N; i++) {
    long long h = trial_hits(i);
    hits += h;
}
```

- With `reduction(+:hits)`, each thread has its own private `hits`
- At the end of the parallel loop, each thread's `hits` is summed into the original `hits`
- Therefore, the shared variable does not need to be updated on every iteration
 - Unlike `atomic`, it avoids updating the shared variable every time, so it tends to be faster

Conceptual View of reduction

- Conceptually, reduction is close to the following:

```
long long hits_local[MAX_THREADS] = {0};

#pragma omp parallel
{
    int tid = omp_get_thread_num();

    #pragma omp for
    for (int i = 0; i < N; i++) {
        long long h = trial_hits(i);
        hits_local[tid] += h;
    }
}

long long hits = 0;
for (int t = 0; t < nthreads; t++) {
    hits += hits_local[t];
}
```

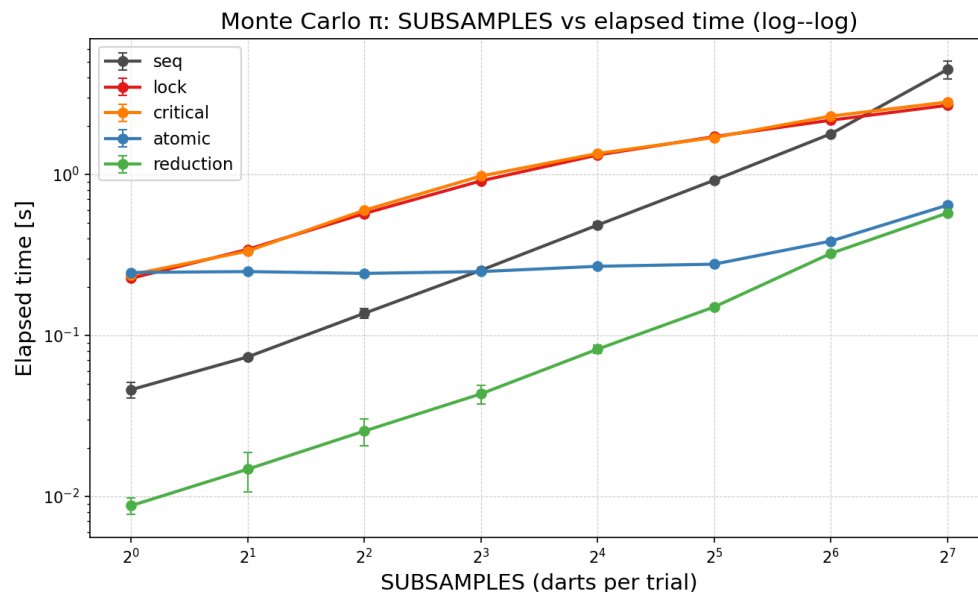
Timing Comparison

- Compare `atomic.c` and `reduction.c` on the same Monte Carlo example
- Example output

```
$ ./atomic  
Elapsed time: 1.273485 seconds  
$ ./reduction  
Elapsed time: 0.340737 seconds
```

Note: Results Are Not Always Like This

- Parallelization benefits depend on the amount of work in the parallel region
- Varying SUBSAMPLES changes the cost of `trial_hits`
 - When small, each iteration is light, so synchronization on hits dominates and the parallel version may be slower than seq
 - When large, `trial_hits(i)` dominates and parallelization is more effective



[Note] Operations Supported by reduction

- reduction supports operations other than +
- In C/C++, + * & | ^ && || max min are available
- Users can also define reduction operations with declare reduction

```
typedef struct {
    double x; double y;
} Pair;

#pragma omp declare reduction( \
    pair_plus : Pair : \
    omp_out.x += omp_in.x, omp_out.y += omp_in.y \
) initializer(omp_priv = {0.0, 0.0})

Pair total = {0.0, 0.0};
#pragma omp parallel for reduction(pair_plus:total)
for (int i = 0; i < N; i++) {
    Pair p = compute_pair(i);
    total.x += p.x;
    total.y += p.y;
}
```

目次

What is OpenMP?

Loop Parallelization

For Correct Parallelization

▸ **For Fast Parallelization**

▸ **Scheduling**

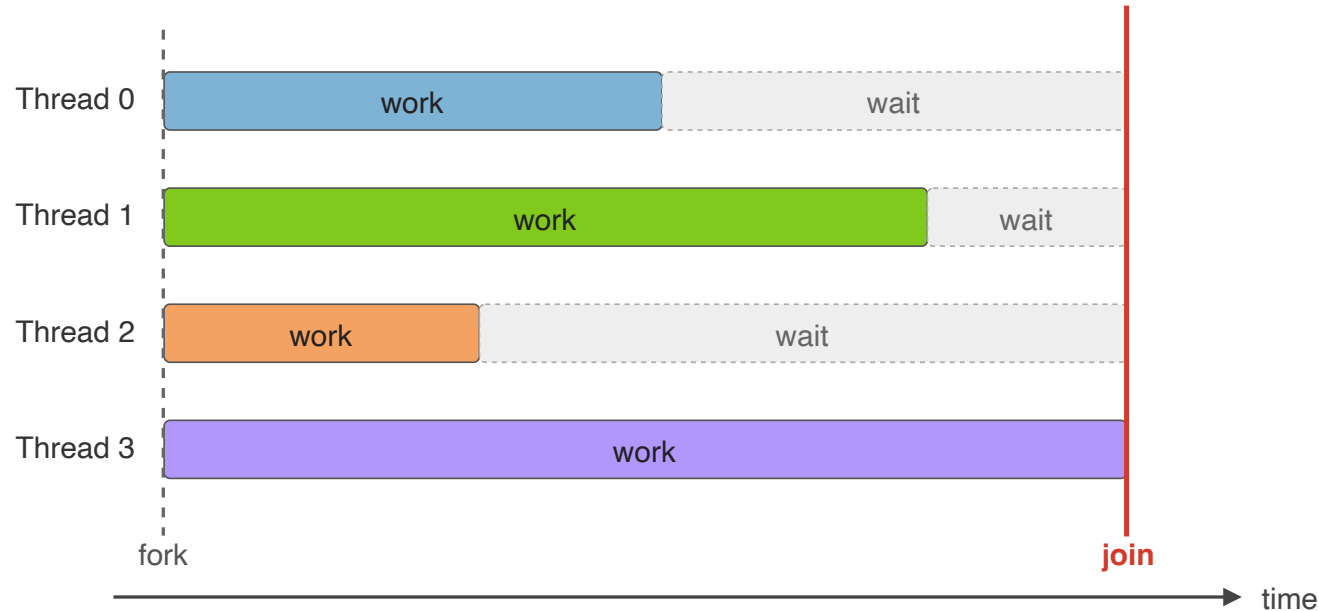
Thread Count Tuning

Other Constructs

Appendix

Importance of Scheduling

- Scheduling is important for high performance
- Even with parallelization, imbalanced load distribution hurts performance
- Especially with OpenMP's fork-join model, all threads wait for the busiest one



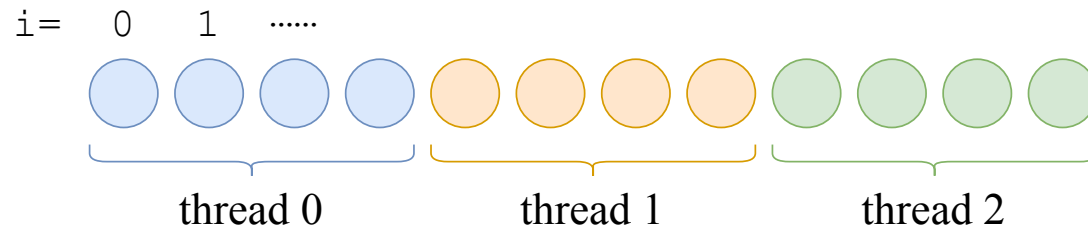
Types of Scheduling

- There are three scheduling types for for loops:
 1. **static**: statically assign chunk iterations per thread
 2. **dynamic**: dynamically assign chunk iterations; idle threads fetch the next chunk
 3. **guided**: gradually decrease the number of iterations assigned per thread
- Specified as `#pragma omp parallel for schedule(dynamic, CHUNK_SIZE)`
 - If omitted, the spec does not mandate a default, but major implementations use static

static

- `schedule(static, CHUNK_SIZE)`

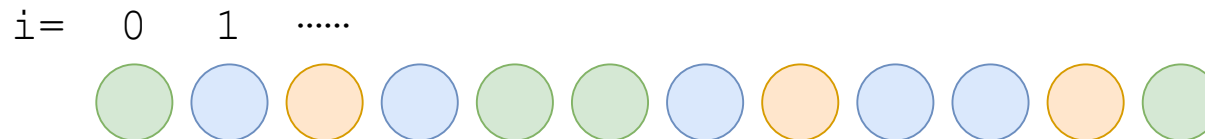
- Split the loop into chunks of `CHUNK_SIZE` and assign them **statically** in round-robin starting from thread 0
- The default when the schedule clause is omitted is static, with `CHUNK_SIZE =  $\frac{\text{number of iterations}}{\text{number of threads}}$` (i.e., this is what you get with no schedule clause)
- When `CHUNK_SIZE =  $\frac{\text{number of iterations}}{\text{number of threads}}$`



- When `CHUNK_SIZE = 1`



- `schedule(dynamic, CHUNK_SIZE)`
 - Split the loop into chunks of `CHUNK_SIZE` and **assign the next chunk to whichever thread finishes**
 - Effective when per-iteration cost is unknown until runtime
 - If `CHUNK_SIZE` is omitted, it defaults to 1



- `schedule(guided, CHUNK_SIZE)`
 - **Gradually decrease chunk size while assigning the next work to finishing threads**
 - `CHUNK_SIZE` is the minimum chunk size
 - If $k > 1$ is specified, chunk size decreases exponentially toward k , but the final chunk may be smaller than k
 - If `CHUNK_SIZE` is omitted, it defaults to 1



Characteristics of Each Scheduling Type

- Chunk size for dynamic and guided affects performance
 - Small chunk → better load balance but higher overhead
 - Large chunk → worse load balance but lower overhead
- Assignment overhead
 - dynamic and guided schedule at runtime, so overhead is higher than static
 - **If you can arrange scheduling so load is balanced in advance,** using static scheduling may be even faster

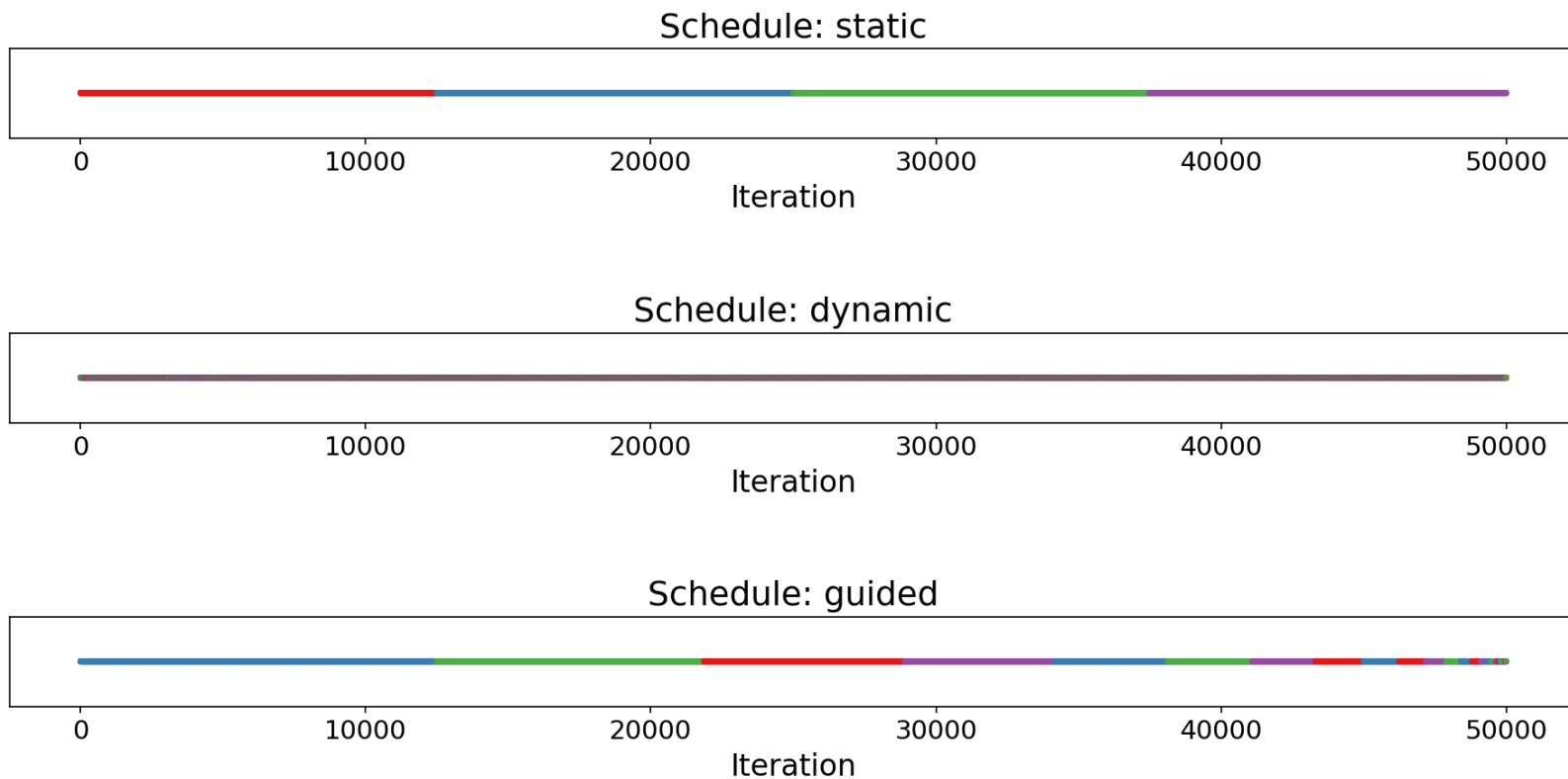
Scheduling Comparison

- Parallelize only the outer for loop below with OpenMP
 - Later iterations have more work
- Try all three scheduling types and compare execution time
- Also record which thread handled which iteration

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < i; j++) {  
        A[i]++;  
    }  
}
```

Scheduling Comparison

- Run `static.c`, `dynamic.c`, and `guided.c` (4 threads)



Scheduling Comparison: Measurement Results

- Example output:

```
=== OpenMP threads: 4 ===  
static : 0.372155 ± 0.004169 s  
dynamic: 0.586862 ± 0.003830 s  
guided : 0.259611 ± 0.067941 s
```

- Despite good load balance, dynamic is slower than static
 - Scheduling overhead is likely the cause
- guided is fastest
 - It balances low overhead with good load balance

Changing static Chunk Size

- Setting static `CHUNK_SIZE` to 1 splits more evenly (note the original chunk size was $\frac{\text{number of iterations}}{\text{number of threads}}$)
- Let's run it

```
=== OpenMP threads: 4 ===  
static : 0.264197 ± 0.042221 s  
dynamic: 0.586862 ± 0.003830 s  
guided  : 0.259611 ± 0.067941 s
```

- Static assignment might have made static faster than guided, but it was not clearly faster
 - Cache hit rate differences?
guided has sequential access early on; static strides by 4

Reference: collapse Directive

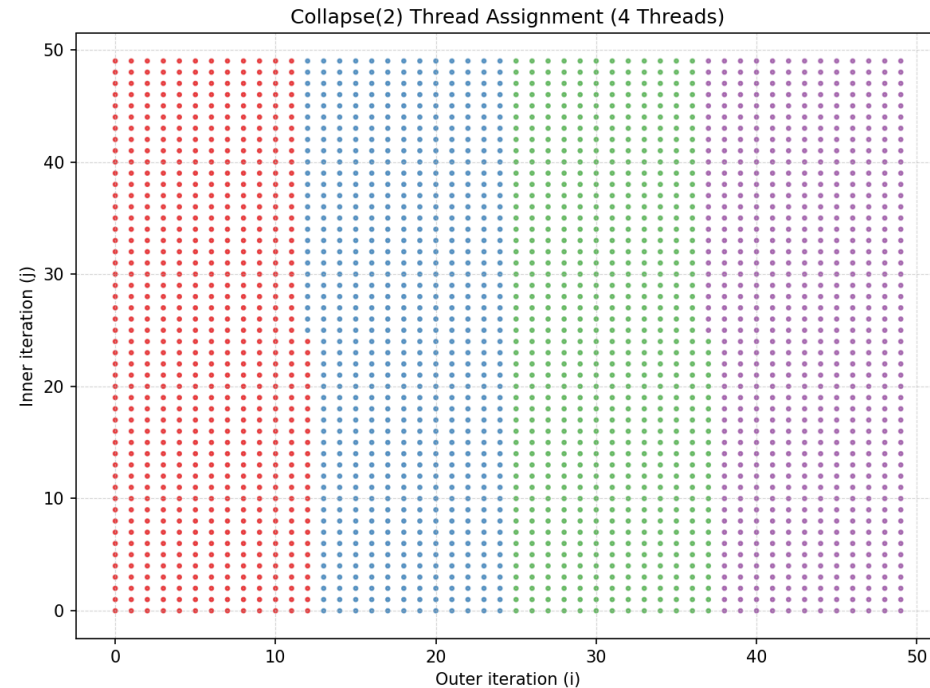
- When parallelizing nested loops, you can use the collapse directive

```
#pragma omp parallel for collapse(2)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        C[i][j] = A[i][j] + B[i][j];
    }
}
```

- No operations may appear between for loops (must be a pure nested loop)
- collapse(n) treats n nested loops as **one large loop** for parallelization
- Can be combined with the schedule directive

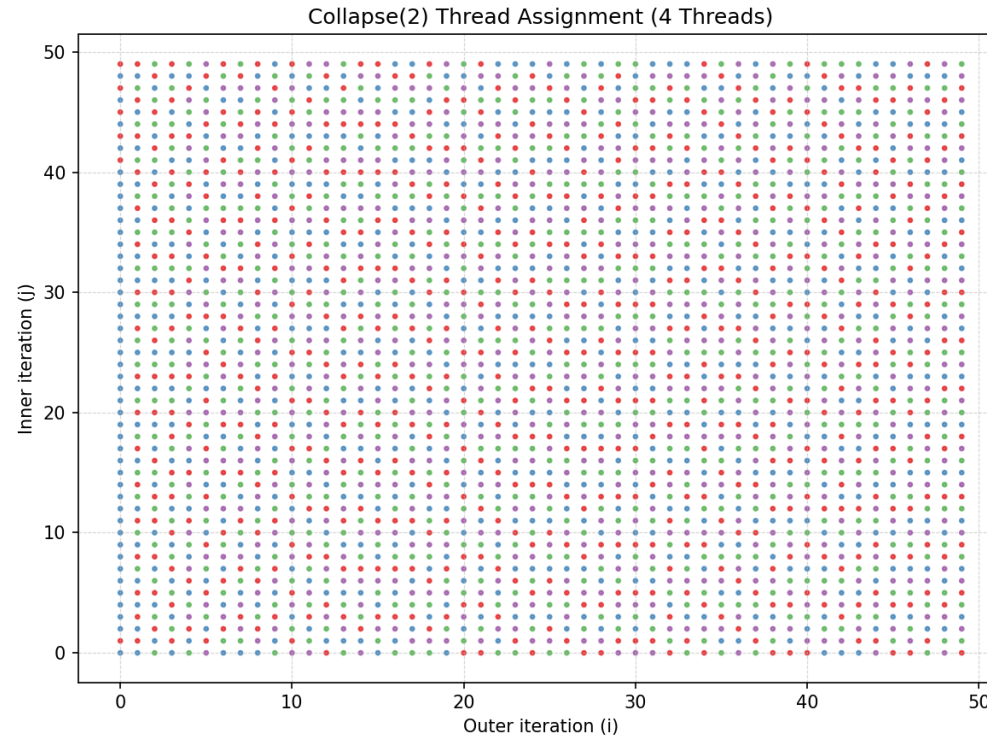
Investigating collapse Behavior

- Run `collapse.c`
- Visualize which iterations each thread handles, color-coded per thread
- Results with no `schedule` clause (defaults to `static`):



Investigating collapse Behavior

- Results with `schedule(dynamic, 1)`:



- The nested-loop structure is no longer relevant

When collapse Is Useful

- Collapsing nested loops increases the number of iterations to distribute
- This may not help much on CPU programs like the above, but **parallelizing for loops on GPUs** is a useful case
- See the Appendix examples for details

目次

What is OpenMP?

Loop Parallelization

For Correct Parallelization

▸ **For Fast Parallelization**

Scheduling

▸ **Thread Count Tuning**

Other Constructs

Appendix

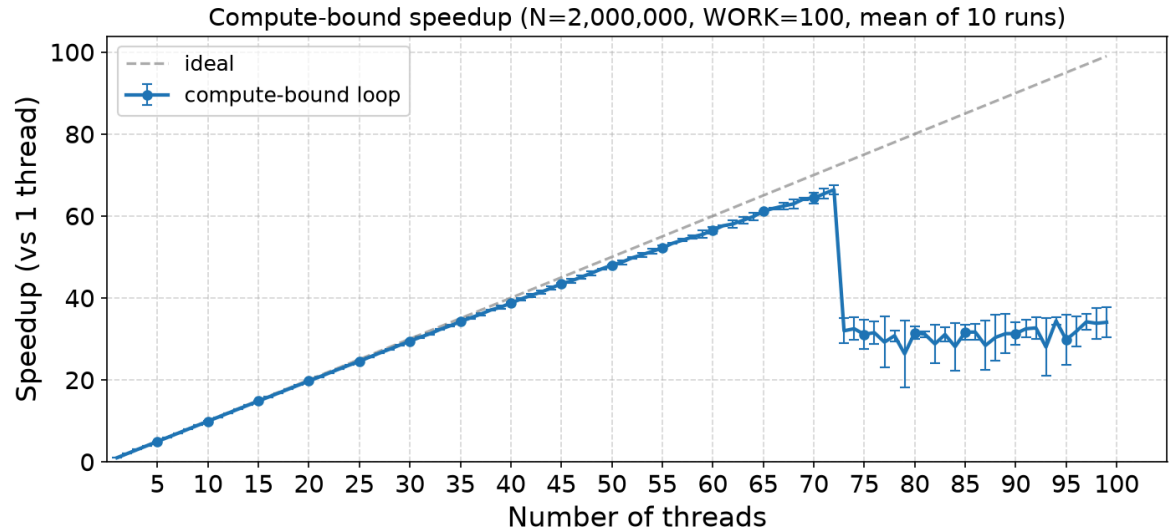
What Is the Optimal Thread Count?

- Changing topics: what is the optimal thread count?
- More threads do not always mean better performance
 - Only up to the **number of logical cores** can run threads at once
- Commands to find available logical cores:
 - Linux: `nproc`
 - macOS: `sysctl -n hw.logicalcpu`
- Miyabi-g has 72 logical cores

Thread Count vs. Runtime ①: Compute-heavy Example

- Run `measure_threads_compute.c`, which does heavy per-element work, with varying thread counts
 - With enough work per iteration, strong speedup is expected

```
WORK = 100
#pragma omp parallel for
for (long i = 0; i < N; i++) {
    double x = 1.0 + 1.0e-9 * i;
    for (int k = 0; k < WORK; k++) {
        x = x * 1.0000001 + 0.0000001;
        x = x * 0.9999999 + 0.0000002;
    }
    out[i] = x;
}
```

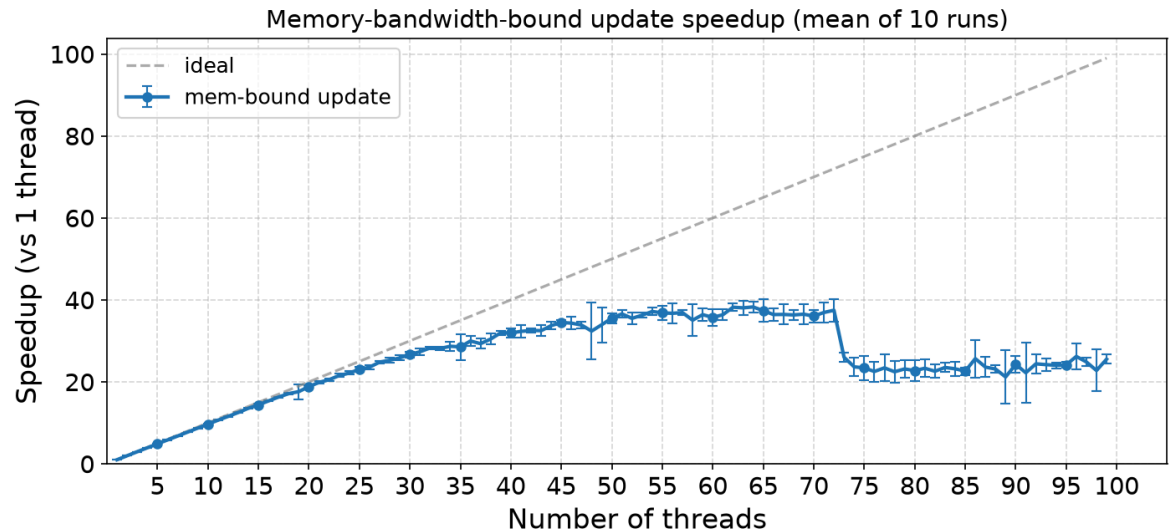


- Speedup grows with thread count but diverges from ideal near the logical core count

Thread Count vs. Runtime ②: Memory Bandwidth Effects

- Increasing threads does not always keep improving performance
- Example: simple operation on large arrays (measure_threads_mem.c)
 - This workload is memory-bound relative to compute
 - Reads $B[i]$, $C[i]$ and writes $A[i]$
- With more threads, **memory bandwidth** may saturate before CPU cores

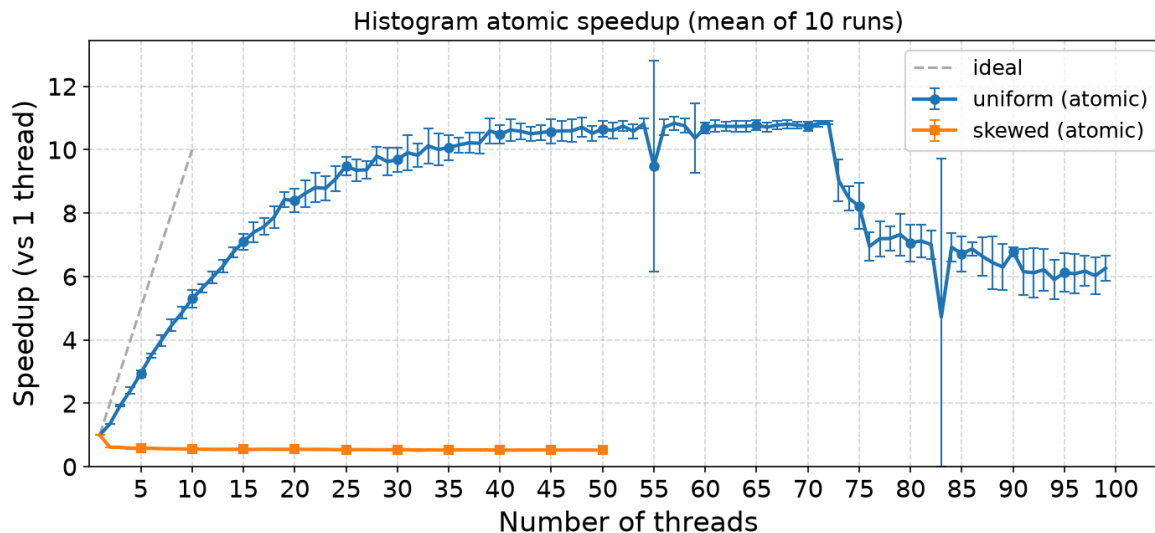
```
#pragma omp parallel for
for (long i = 0; i < N; i++) {
    A[i] = B[i] + 2.0 * C[i];
}
```



Thread Count vs. Runtime ③: Contention Effects

- Even with correct synchronization, more threads are not always faster
- Example: histogram computation (measure_threads_hist.c)
- `atomic` prevents data races
- However, if many threads frequently update the same `hist[bin]`, that location becomes a bottleneck
- If data is skewed toward certain bins, contention worsens with more threads

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    int bin = data[i];
    #pragma omp atomic
    hist[bin]++;
}
```



目次

What is OpenMP?

Loop Parallelization

For Correct Parallelization

For Fast Parallelization

▸ **Other Constructs**

Appendix

single Construct

- Used to run code on **only one thread** inside a parallel region
- Inside single, only one thread executes
- Which thread runs is unspecified
- Unless nowait is used, **there is an implicit barrier after single**
- See also single.c

```
#pragma omp parallel
{
    #pragma omp single
    {
        work();
    }
}
```

masked Construct

- Similar to `single`; only some threads in the team execute
- Without a clause, **thread 0 executes**
 - The `filter` clause specifies which threads execute
- **There is no implicit barrier afterward**
- See also `masked.c`
 - The older `master` construct is deprecated; OpenMP 5.1 recommends `masked`

```
#pragma omp parallel
{
    #pragma omp masked
    {
        work();
    }
}
```

Non-loop Parallelization ①: sections Construct

- A somewhat more flexible(?) parallelization method than for
- Each section is assigned to a thread

```
#pragma omp parallel sections
{
    #pragma omp section
    {
        work1();
    }
    #pragma omp section
    {
        work2();
    }
}
```

- Details omitted

Non-loop Parallelization ②: task Construct

- A construct for so-called task parallelism
- So far we mainly distributed for loop iterations across threads
- However, work to parallelize is not always expressible as a for loop
 - Recursive divide-and-conquer
 - Tree/graph traversal
 - Work that appears dynamically at runtime
- The task construct allows more flexible parallel units

Non-loop Parallelization ②: task Construct

- Example: computing the Fibonacci sequence

```
int fib(int n) {
    if (n < 2) return n;
    else{
        #pragma omp task shared(x) firstprivate(n)
        x = fib(n - 1);
        #pragma omp task shared(y) firstprivate(n)
        y = fib(n - 2);
        #pragma omp taskwait
        return x + y;
    }
}

#pragma omp parallel
{
    #pragma omp single
    printf("Fibonacci of %d is %d\n", 10, fib(10));
}
```

- **task** creates work that “someone can execute later”
 - Note: task does not create new threads
 - Generated tasks are scheduled onto existing threads by the OpenMP runtime
- **taskwait** waits for child tasks to complete

Non-loop Parallelization ②: task Construct

- Example: computing the Fibonacci sequence

```
int fib(int n) {  
    if (n < 2) return n;  
    else{  
        #pragma omp task shared(x) firstprivate(n)  
        x = fib(n - 1);  
        #pragma omp task shared(y) firstprivate(n)  
        y = fib(n - 2);  
        #pragma omp taskwait  
        return x + y;  
    }  
}  
  
#pragma omp parallel  
{  
    #pragma omp single  
    printf("Fibonacci of %d is %d\n", 10, fib(10));  
}
```

- If all threads call `fib(10)`, the same work is duplicated
- So only one thread creates the initial tasks via `single` or `masked`
- Idle threads can also execute generated tasks

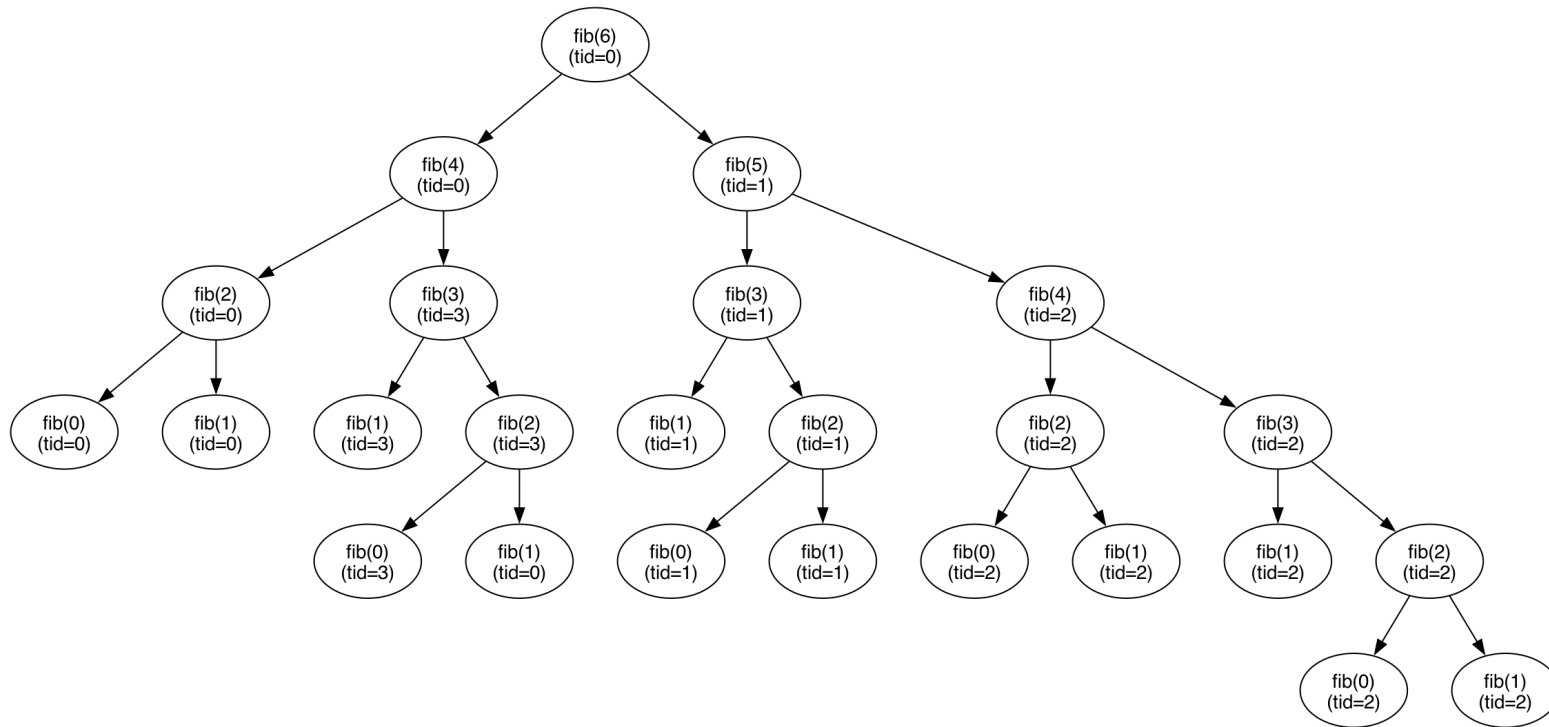
Role of taskwait

- `taskwait` waits for child tasks created by the current task
- In the Fibonacci example, we must wait until `fib(n - 1)` and `fib(n - 2)` finish
- Without `taskwait`, `x + y` may be returned before `x` and `y` are computed

```
#pragma omp task shared(x) firstprivate(n)
x = fib(n - 1);
#pragma omp task shared(y) firstprivate(n)
y = fib(n - 2);
#pragma omp taskwait
return x + y;
```

Visualizing Task Execution

- Run `fib_tree.c` and visualize task creation with `animate_fib.py`



- [Link to GIF](#)

[Note] Caveats of the Fibonacci Example

- Fibonacci is easy to explain for illustrating task
- But it is not a good example of practical high-performance optimization
 - There is no real need for task parallelism here
 - Tasks are too fine-grained; creation overhead dominates
 - It does not resemble real task-parallel programs
- It is often used to measure runtime overhead instead
- For more realistic task-parallel benchmarks, see BOTS

Controlling Task Granularity

- Creating and managing tasks has overhead
 - If tasks are too small, sequential execution is faster than taskifying
- In practice, a threshold (**cutoff**) like this is common

```
int fib(int n) {  
    if (n < 20) {  
        return fib_serial(n);  
    }  
  
    int x, y;  
    #pragma omp task shared(x) firstprivate(n)  
    x = fib(n - 1);  
    #pragma omp task shared(y) firstprivate(n)  
    y = fib(n - 2);  
    #pragma omp taskwait  
    return x + y;  
}
```

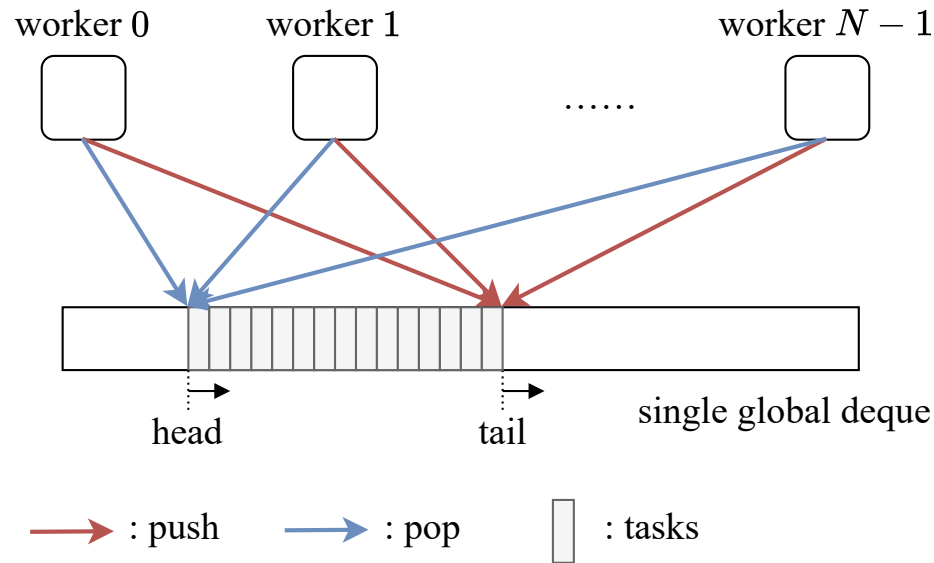
if Clause

- OpenMP task can take an if clause
- If true, a normal task is created
- If false, the task is generally executed immediately in place
- Like a manual cutoff, it avoids creating overly fine tasks

```
#pragma omp task if(n > 20) shared(x) firstprivate(n)  
x = fib(n - 1);  
#pragma omp task if(n > 20) shared(y) firstprivate(n)  
y = fib(n - 2);
```

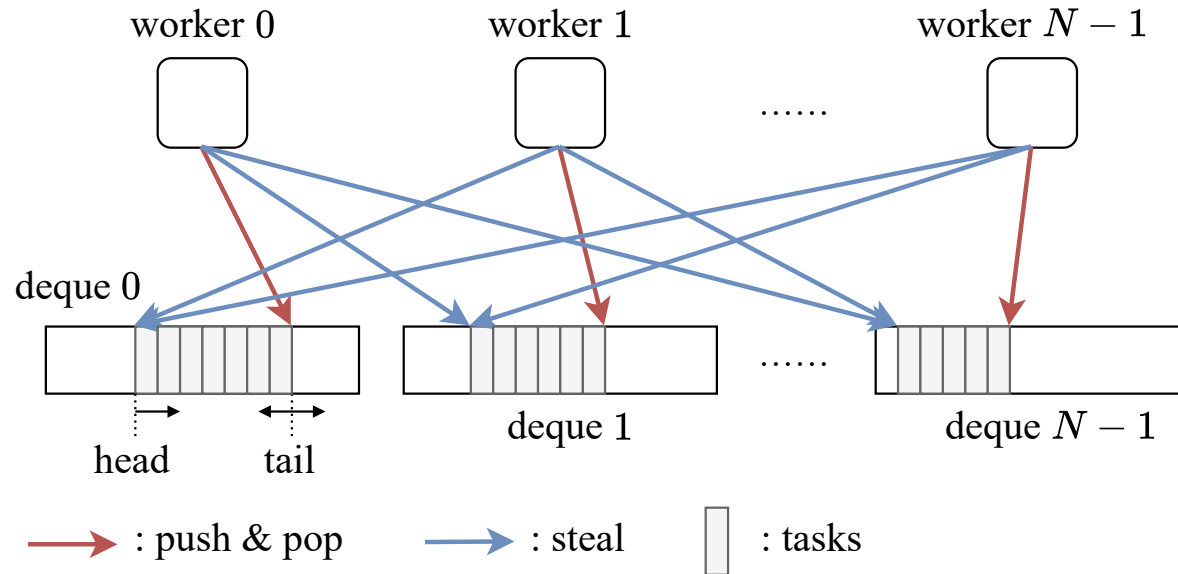
[Note] Task Parallel Scheduling (1/3)

- Task parallelism usually uses **dynamic load balancing** rather than **static** due to irregularity
- The simplest approach is a shared queue
 - One shared task queue (queue of runnable tasks) all workers push/pop tasks
 - High contention on queue access



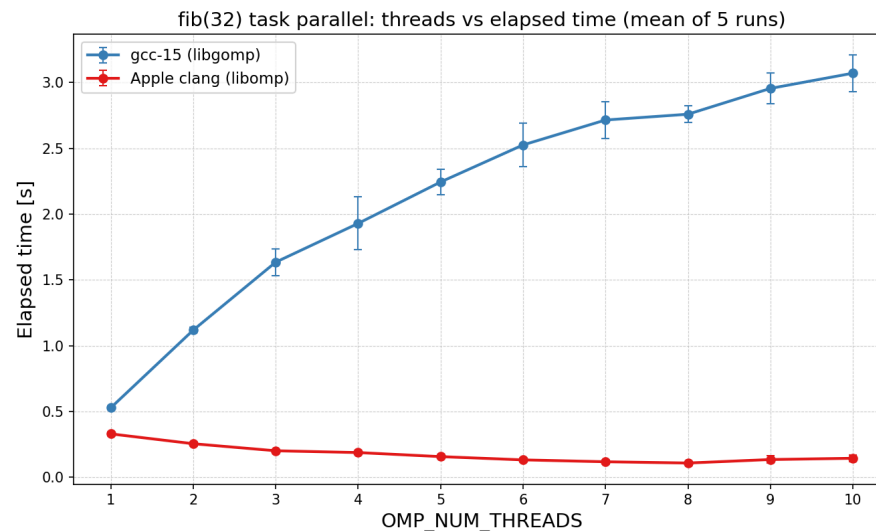
[Note] Task Parallel Scheduling (2/3)

- A faster dynamic approach is **work stealing**
- Each worker has its own task queue
 - When its queue is empty, it steals tasks from others
 - Known to **scale better as worker count grows** by reducing queue contention



[Note] Task Parallel Scheduling (3/3)

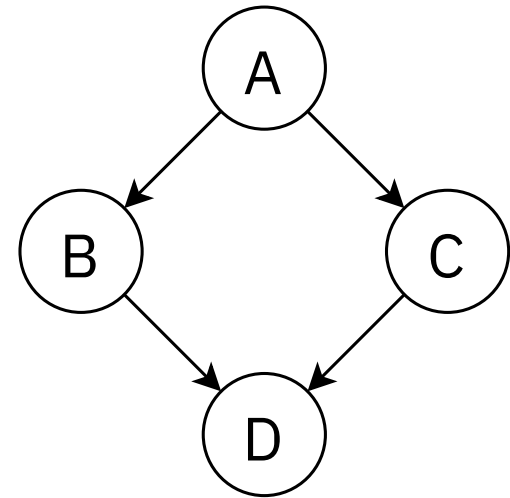
- The OpenMP spec does not fix scheduling policy; it varies by implementation
 - GCC libgomp: shared queue
 - LLVM libomp: work stealing
- Measure while scaling OMP_NUM_THREADS on fib
- LLVM libomp scales well; GCC libgomp slows down with more threads



[Note] task depend

- The depend clause specifies dependencies between tasks
- Dependent work can be expressed as a DAG
 - Tasks run when dependencies are satisfied
 - Enables finer control than plain fork-join synchronization

```
#pragma omp task depend(out: A)  
compute_A();  
#pragma omp task depend(in: A) depend(out: B)  
compute_B();  
#pragma omp task depend(in: A) depend(out: C)  
compute_C();  
#pragma omp task depend(in: B, C)  
compute_D();
```



[Note] Distributed-memory Task Parallel System Itoyori

- OpenMP task is task parallelism in shared memory
- Task parallelism across distributed memory on multiple nodes is not easy
 - Which node runs each task
 - Where to place required data
 - When to communicate across nodes
 - How to balance load
- Itoyori is one example of a **runtime for task parallelism in distributed memory** ([Paper](#), [GitHub](#))

[Note] GPU Task Parallel System GTaP

- GTaP is a runtime for task parallelism on GPUs ([Paper](#), [GitHub](#))
- A resident GPU runtime manages task creation, execution, and synchronization
 - Schedules tasks on the GPU instead of launching many small kernels from the CPU
 - `#pragma gtap task` / `#pragma gtap taskwait` allow OpenMP-like syntax
 - Supporting fork-join on GPUs is not trivial
- Two execution modes:
 - **1 thread 1 task**: Run fine-grained irregular tasks on many GPU threads
 - **1 thread block 1 task**: When a task has internal data parallelism, threads in one block cooperate

目次

What is OpenMP?

Loop Parallelization

For Correct Parallelization

For Fast Parallelization

Other Constructs

▸ **Appendix**

▸ **SIMD**

OpenMP GPU Support

Program Examples

Implementation of Directives

SIMD Features in OpenMP

- SIMD runs the same instruction on multiple data elements in parallel
- OpenMP provides SIMD via `#pragma omp simd`

```
#pragma omp simd  
for (int i = 0; i < N; i++) {  
    b[i] = 2 * a[i];  
}
```

Reference: Compilation and Optimization Report

- Compile as below to write a vectorization report to `vec_report.txt`
- `-O3` sets the optimization level

```
$ gcc-15 -O3 -fopt-info-vec-optimized=vec_report.txt -fopenmp simd.c -o simd
```

- Example output:

```
simd.c:30:21: optimized: loop vectorized using 16 byte vectors  
simd.c:30:21: optimized: loop vectorized using 8 byte vectors  
simd.c:18:10: optimized: loop vectorized using 16 byte vectors
```

- This confirms SIMD vectorization

Reference: Generating Assembly

- Compile as below to emit assembly to `simd.s`

```
$ gcc-15 -O1 -fopenmp -S simd.c -o simd.s
```

- Example output:

```
ldr q31, [x7, x1]           // 128-bit load from address x7 + x1 (four ints)
add v31.4s, v31.4s, v31.4s  // double each of four 32-bit ints in v31 (vector add)
str q31, [x0, x1]           // 128-bit store to x0 + x1 (write result)
```

- Data is loaded into vector registers and four integers are doubled at once
- On Mac, the SIMD extension appears to be Arm Neon
- SIMD registers are 128 bits wide, so four 32-bit ints or two 64-bit floats can be processed at once

Runtime Comparison With and Without SIMD

- make bench-simd runs each configuration 10 times and shows mean $\pm$ std dev
 - At -O1, with and without #pragma omp simd
 - Confirm SIMD instructions are enabled/disabled in assembly
- With SIMD:

```
$ make bench-simd
# SIMD (simd), 10 runs each
OMP_NUM_THREADS=1: 0.032581  $\pm$  0.000935
OMP_NUM_THREADS=4: 0.015092  $\pm$  0.000373
```

- Without SIMD:

```
# scalar (simd_scalar), 10 runs each
OMP_NUM_THREADS=1: 0.048844  $\pm$  0.001819
OMP_NUM_THREADS=4: 0.017411  $\pm$  0.000664
```

Runtime Comparison With and Without SIMD

- Above: mean over 10 runs per configuration; std dev included
- However, SIMD is not always faster by exactly the vector width
 - Memory access, cache, loop control, and instruction scheduling matter
 - With more threads, memory bandwidth and other factors may bottleneck
- In this environment/program, at -O3, SIMD was enabled even without `#pragma omp simd`

目次

What is OpenMP?

Loop Parallelization

For Correct Parallelization

For Fast Parallelization

Other Constructs

▸ **Appendix**

SIMD

▸ **OpenMP GPU Support**

Program Examples

Implementation of Directives

OpenMP Support for GPUs

- Since OpenMP 4.0, features for accelerators such as GPUs were added
- The target directive enables GPU execution
- A similar accelerator API is **OpenACC**
 - ACC stands for Accelerator
 - Unlike OpenMP, designed specifically for accelerators

Basic Model of OpenMP GPU Offloading

```
#pragma omp target teams distribute parallel for \  
           map(to: A[0:N], B[0:N]) map(from: C[0:N])  
for (int i = 0; i < N; i++) {  
    C[i] = A[i] + B[i];  
}
```

- **target**: execute on the GPU
- **teams**: create multiple teams on the GPU
- **distribute**: distribute loop iterations among teams
- **parallel for**: parallelize the loop within each team
- **map**: transfer data between host and device

GPU Caveat: Data Transfer Cost

- GPU compute may not help if data is transferred every time
 - Transfer input from host to device memory
 - Compute on device
 - Transfer results from device to host memory
- Many small GPU jobs make transfer/launch overhead dominate compute time

```
#pragma omp target data map(to: A[0:N], B[0:N]) map(from: C[0:N])
{
    #pragma omp target teams distribute parallel for
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }

    // Run multiple GPU kernels here if needed
}
```

目次

What is OpenMP?

Loop Parallelization

For Correct Parallelization

For Fast Parallelization

Other Constructs

▸ **Appendix**

SIMD

OpenMP GPU Support

▸ **Program Examples**

Implementation of Directives

for Parallel Example: Matrix Multiplication

- Tuned matrix multiplication with OpenMP for parallelism
 - Loop reordering (i-k-j) for sequential access to rows of A and B
 - Blocking (TILE) for L1/L2 cache reuse
 - Parallelize the outer loop with `#pragma omp parallel for`
- See `matmul_tuned.c`

```
#pragma omp parallel for schedule(static)
for (int ii = 0; ii < N; ii += TILE) {
    for (int kk = 0; kk < N; kk += TILE) {
        for (int jj = 0; jj < N; jj += TILE) {
            for (int i = ii; i < i_end; i++) {
                for (int k = kk; k < k_end; k++) {
                    double a_ik = A[i][k];
                    for (int j = jj; j < j_end; j++) {
                        C[i][j] += a_ik * B[k][j];
                    }
                }
            }
        }
    }
}
```


task Parallel Example: merge sort, cilk sort

- Recursive sort pairs well with OpenMP task
 - Split the array and taskify sorting subarrays
 - Sort small subarrays sequentially without tasks
 - e.g., insertion sort, sequential merge sort
 - `taskwait` for children, then merge
- Simple parallel merge sort:
 - Taskify sorting left and right halves
 - Merge sequentially at the end
- Cilk sort:
 - **Taskify merge recursively, not just sort**
 - Extracts more parallelism
- See `parallel_merge_sort.c` and `cilk_sort.c`

GPU Execution Example: Matrix Multiplication

- GPU matrix multiplication in OpenMP
 - Simple, no extra tuning
- See `matmul_gpu.c`

```
#pragma omp target data map(to:A[0:N][0:N], B[0:N][0:N]) map(from:C[0:N][0:N])
{
    #pragma omp target teams distribute parallel for collapse(2)
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            for(int k = 0; k < N; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

GPU Execution Example: Matrix Multiplication

- Compare runtime with and without `collapse(2)` (on miyabi-g)
- Example output:

```
$ ./matmul_gpu  
Use collapse  
Test passed!  
Computation time: 0.386624 seconds
```

```
$ ./matmul_gpu  
No collapse  
Test passed!  
Computation time: 0.702773 seconds
```

- `collapse` increases iterations from N to N^2 , spreading work across many GPU threads

目次

What is OpenMP?

Loop Parallelization

For Correct Parallelization

For Fast Parallelization

Other Constructs

▸ **Appendix**

SIMD

OpenMP GPU Support

Program Examples

▸ **Implementation of Directives**

Implementation of Directives

- May contain errors
- Below we focus only on `libgomp` in `gcc`
- See <https://github.com/gcc-mirror/gcc/tree/releases/gcc-9.2.0/libgomp>

Directive Implementation ①: Thread Creation

- The `parallel` directive expands roughly as:

```
void subfunction (void *data)
{
    use data;
    body;
}

setup data;
GOMP_parallel_start (subfunction, &data, num_threads);
subfunction (&data);
GOMP_parallel_end ();
```

- `GOMP_parallel_start` calls `gomp_team_start()` to create OS threads (Linux, Mac: `pthread_create`)
- See `parallel.c` and `team.c` in the repo

Directive Implementation ②: lock Implementation

- In my investigation, equivalent to pthread_mutex
- Defined in config/posix/mutex.h as:

```
typedef pthread_mutex_t gomp_mutex_t;  
static inline void gomp_mutex_init (gomp_mutex_t *mutex)  
{  
    pthread_mutex_init (mutex, NULL);  
}  
static inline void gomp_mutex_lock (gomp_mutex_t *mutex)  
{  
    pthread_mutex_lock (mutex);  
}  
// omitted
```

- See lock.c

Directive Implementation ③: critical Implementation

- Unnamed critical section definition

```
void
GOMP_critical_start (void)
{
    /* There is an implicit flush on entry to a critical region. */
    __atomic_thread_fence (MEMMODEL_RELEASE);
    gomp_mutex_lock (&default_lock);
}

void
GOMP_critical_end (void)
{
    gomp_mutex_unlock (&default_lock);
}
```

- Apparently managed with a mutex
- See `critical.c`

Directive Implementation ④: atomic Implementation

- Assembly of `atomic1.c` shows the atomic part as:

```
ldadd w1, w0, [x0]
```

- On ARM, `ldadd` atomically loads from `[x0]`, adds `w0`, stores to `[x0]`, and returns the old value in `w1`

Directive Implementation ④: atomic Implementation

- What about a more complex case?
- Assembly of `atomic2.c` shows:

```
LFB0:
    fadd    d0, d0, d0      // double b
    ldr     x1, [x0]        // load *a into x1
L2:
    fmov     d31, x1        // move x1 to FP register d31
    mov     x2, x1          // copy x1 to x2
    fadd     d31, d0, d31    // d31 = *a + 2*b
    fmov     x3, d31        // store new value in x3
    cas     x2, x3, [x0]    // if [x0] still equals x2 (old), atomically write x3 (new)
                                // in any case, old memory value returned in x2
    cmp     x1, x2          // if x1==x2 after cas, success
    bne     L3              // retry on failure
    ret
```

- This is a CAS loop